

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: G.Kavya

Roll No.: A24126552082

Date: 30-08-2026

TITLE

Interactive To-Do List Using JavaScript, DOM Manipulation and Event Listeners

AIM

To develop an interactive To-Do List using JavaScript, DOM manipulation and event listeners that allows users to add, complete, and delete tasks dynamically.

OBJECTIVES

- To provide a text box for entering a task.
- To provide an Add Task button.
- To dynamically create and add new task items to a list.
- To provide Complete and Delete buttons for each task.
- To visually indicate when a task has been completed.
- To remove the corresponding task from the webpage when Delete is clicked.
- To display an appropriate message when the task list is empty.
- To apply basic CSS styling to the interactive webpage.

EXPECTED LEARNING

- DOM element selection
- Dynamic element creation
- Dynamic modification of the DOM
- Event listeners
- Handling click events
- Adding and removing elements
- Interactive webpage development

TASK DESCRIPTION

Develop an interactive To-Do List application using JavaScript, DOM manipulation and event listeners.

The application should:

- Provide a text box for entering a task.
 - Provide an Add Task button.
 - When the button is clicked, dynamically create a new task item and add it to a list.
 - Each task should contain a Complete button.
 - When the Complete button is clicked, the task should visually indicate that it has been completed.
 - Each task should also contain a Delete button.
 - When Delete is clicked, the corresponding task should be removed from the webpage.
 - Display an appropriate message when the task list is empty.
-

ALGORITHM / PROCEDURE

1. Create an HTML document with a text box, Add Task button, task list, and empty-list message.
2. Select the task input, Add Task button, task list, and empty message using getElementById().
3. Create a function to update the empty-list state.
4. Read and trim the task text entered by the user.
5. If the task text is empty, focus the input field and stop the operation.
6. Create a new list item dynamically using createElement().
7. Create a span element to display the task text.
8. Create a container for the task action buttons.
9. Create a Complete button and attach a click event listener.
10. Toggle completed styling and change the button text when Complete is clicked.
11. Create a Delete button and attach a click event listener.
12. Remove the task item when Delete is clicked.
13. Append the buttons and task text to the task item.
14. Append the task item to the task list.
15. Clear and refocus the input field.
16. Update the empty-list message after adding or deleting tasks.
17. Allow the Enter key to add a task.

PROGRAM / SOURCE CODE — Index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>My To-Do List</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <div class="todo-container">
      <h1>My To-Do List</h1>

      <div class="todo-form">
        <input
          id="taskInput"
          type="text"
          placeholder="Enter a task..."
          aria-label="Enter a task"
        />
        <button id="addTaskBtn" type="button">Add Task</button>
      </div>

      <ul id="taskList" class="task-list" aria-live="polite"></ul>
      <p id="emptyMessage" class="empty-message">No tasks available.</p>
    </div>

    <script src="script.js"></script>
  </body>
</html>
```

Styles.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: linear-gradient(135deg, #f5f7ff, #eefbf4);
  color: #1f2937;
}

.todo-container {
  width: min(540px, 90vw);
  background: #ffffff;
  padding: 32px 24px;
  border-radius: 18px;
  box-shadow: 0 15px 35px rgba(59, 130, 246, 0.12);
}

h1 {
  margin: 0 0 20px;
  text-align: center;
  color: #1d4ed8;
  font-size: clamp(2rem, 3vw, 2.6rem);
}

.todo-form {
  display: flex;
  gap: 12px;
  margin-bottom: 18px;
}

.todo-form input {
  flex: 1;
  padding: 12px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
  font-size: 1rem;
}

.todo-form input:focus {
  outline: 2px solid #93c5fd;
  border-color: #60a5fa;
}

button {
  border: none;
  border-radius: 10px;
  padding: 10px 16px;
  font-weight: 600;
  cursor: pointer;
  transition: transform 0.2s ease, opacity 0.2s ease;
}

button:hover {
  transform: translateY(-1px);
}

#addTaskBtn {
  background: linear-gradient(90deg, #2563eb, #3b82f6);
  color: #fff;
}

.task-list {
  list-style: none;
  padding: 0;
  margin: 0;
}

.task-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  padding: 12px 14px;
  margin-bottom: 10px;
  background: #f8fafc;
}
```

```

border: 1px solid #e2e8f0;
border-radius: 10px;
}

.task-text {
  flex: 1;
  word-break: break-word;
  color: #0f172a;
}

.completed {
  background: #ecfdf5;
  border-color: #86efac;
}

.completed-text {
  text-decoration: line-through;
  color: #16a34a;
}

.task-actions {
  display: flex;
  gap: 8px;
}

.complete-btn {
  background: #dcfce7;
  color: #166534;
}

.delete-btn {
  background: #fee2e2;
  color: #b91c1c;
}

.empty-message {
  margin: 0;
  padding: 16px;
  text-align: center;
  color: #64748b;
  background: #f8fafc;
  border: 1px dashed #cbd5e1;
  border-radius: 10px;
}

@media (max-width: 560px) {
  .todo-form,
  .task-item {
    flex-direction: column;
    align-items: stretch;
  }

  .task-actions {
    width: 100%;
  }

  .task-actions button {
    flex: 1;
  }
}

```

Script.js

```
const taskInput = document.getElementById('taskInput');
const addTaskBtn = document.getElementById('addTaskBtn');
const taskList = document.getElementById('taskList');
const emptyMessage = document.getElementById('emptyMessage');

function updateEmptyState() {
  const tasks = taskList.querySelectorAll('.task-item');
  const hasTasks = tasks.length > 0;

  taskList.style.display = hasTasks ? 'block' : 'none';
  emptyMessage.style.display = hasTasks ? 'none' : 'block';
}

function addTask(taskText) {
  const text = taskText.trim();

  if (!text) {
    taskInput.focus();
    return;
  }

  const listItem = document.createElement('li');
  listItem.className = 'task-item';

  const taskTextElement = document.createElement('span');
  taskTextElement.className = 'task-text';
  taskTextElement.textContent = text;

  const actionGroup = document.createElement('div');
  actionGroup.className = 'task-actions';

  const completeBtn = document.createElement('button');
  completeBtn.className = 'complete-btn';
  completeBtn.type = 'button';
  completeBtn.textContent = 'Complete';

  completeBtn.addEventListener('click', () => {
    listItem.classList.toggle('completed');
    taskTextElement.classList.toggle('completed-text');

    const isCompleted = listItem.classList.contains('completed');
    completeBtn.textContent = isCompleted ? 'Completed' : 'Complete';
  });

  const deleteBtn = document.createElement('button');
  deleteBtn.className = 'delete-btn';
  deleteBtn.type = 'button';
  deleteBtn.textContent = 'Delete';

  deleteBtn.addEventListener('click', () => {
    listItem.remove();
    updateEmptyState();
  });

  actionGroup.appendChild(completeBtn);
  actionGroup.appendChild(deleteBtn);

  listItem.appendChild(taskTextElement);
  listItem.appendChild(actionGroup);
  taskList.appendChild(listItem);

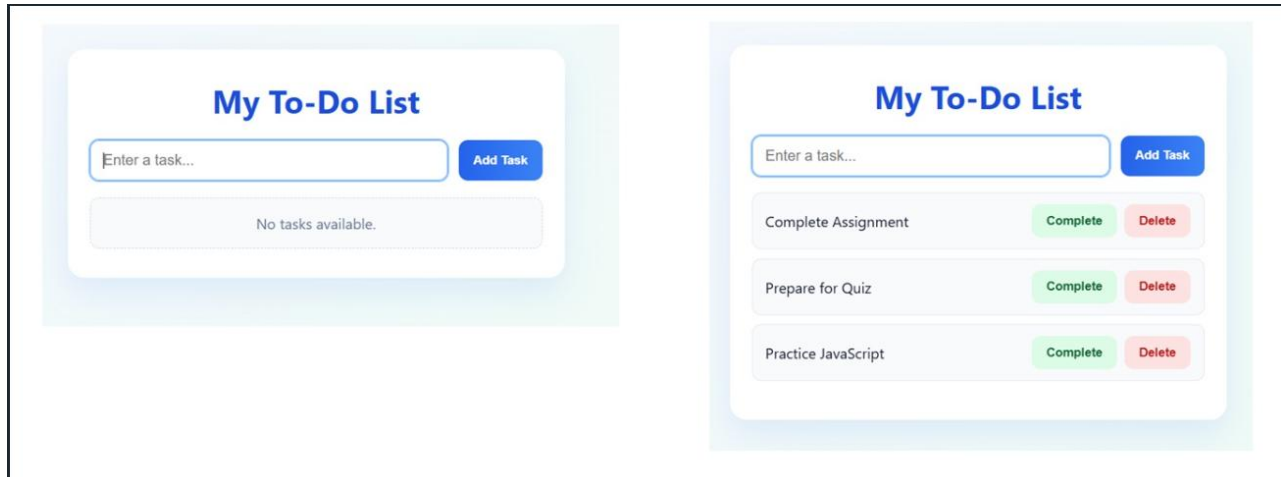
  taskInput.value = '';
  taskInput.focus();
  updateEmptyState();
}

addTaskBtn.addEventListener('click', () => {
  addTask(taskInput.value);
});

taskInput.addEventListener('keydown', (event) => {
  if (event.key === 'Enter') {
    addTask(taskInput.value);
  }
});

updateEmptyState();
```

OUTPUT:



CODE EXPLANATION

Component	Purpose
<code>getElementById()</code>	Selects HTML elements from the webpage.
<code>updateEmptyState()</code>	Shows or hides the empty-list message.
<code>querySelectorAll()</code>	Finds all task items in the list.
<code>addTask()</code>	Creates and adds a new task dynamically.
<code>.trim()</code>	Removes unnecessary spaces from task text.
<code>createElement()</code>	Creates HTML elements dynamically.
<code>textContent</code>	Sets the text content of an element.
<code>addEventListener()</code>	Handles button and keyboard events.
<code>classList.toggle()</code>	Adds or removes completed styling.
<code>contains()</code>	Checks whether a task is marked completed.
<code>appendChild()</code>	Adds created elements to the webpage.
<code>remove()</code>	Removes the selected task from the webpage.
<code>focus()</code>	Places the cursor back in the task input.
Enter key	Allows a task to be added by pressing Enter.
<code>:hover</code>	Adds a visual effect when hovering over buttons.
<code>@media</code>	Adjusts the layout for smaller screens.

TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Observed Result / Status
TC-01	Open the webpage	My To-Do List and empty message should be displayed	Page displayed correctly — Pass
TC-02	Click Add Task with empty input	Input should remain focused and no task should be added	No task added — Pass

TC-03	Add 'Complete Assignment'	New task item with Complete and Delete buttons should appear	Task added correctly — Pass
TC-04	Add 'Prepare for Quiz' and 'Practice JavaScript'	All entered tasks should appear in the list	Tasks displayed correctly — Pass
TC-05	Click Complete	Task should visually indicate completion and button text should change	Task marked completed — Pass
TC-06	Click Delete	Corresponding task should be removed	Task deleted correctly — Pass
TC-07	Delete all tasks	No tasks available message should be displayed	Empty message displayed — Pass
TC-08	Press Enter after entering a task	Task should be added to the list	Task added using Enter — Pass
TC-09	Open webpage on smaller screen	Layout should adjust to screen size	Responsive layout displayed — Pass

RESULT

Thus, the interactive To-Do List was successfully developed using JavaScript, DOM manipulation, and event listeners. Tasks can be dynamically added, completed, and deleted, and an appropriate message is displayed when the task list is empty. Basic CSS styling and responsive design were also implemented.